

16.

O código imprime uma matriz 3x4 com os números de 1 a 12, isto com os números crescendo durante o “percurso” de cada linha.

17.

a) Ele inverte a string ao contrario

b) Ele inverte a string ao contrário. Seria a mesma coisa se os caracteres ‘)’ e ‘(’ não fossem invertidos.

18.

a)

$p++$ → estamos apontando para o próximo elemento do endereço do nosso ponteiro, ou seja, estamos incrementando 1 em nossa memória (o que muda em bytes de acordo com o tipo de variável que estamos utilizando)

$(*p)++$ → o valor apontado por ‘p’ é incrementado em um

$*(p++)$ → estamos acessando o valor do elemento seguinte no nosso ponteiro (??)

$*(p+10)$ → estamos acessando o valor do decimo elemento seguinte no nosso ponteiro

23.

a) Após a atribuição $pa = \&a[0]$; pa e a possuem valores idênticos, isto é, apontam para o mesmo endereço;

Verdade, pois pa está pegando o endereço do vetor a partir do primeiro termo.

b) A atribuição $pa = \&a[0]$; pode ser escrita como $pa = a$;

Verdade, o ponteiro pega o endereço da mesma forma.

c) $a[i]$ pode ser escrito como $*(a + i)$;

Verdade, se quisermos pegar um valor é indiferente uma das duas formas.

d) $\&a[i]$ e $a + i$ são idênticos;

Verdade, pois estamos pegando o endereço do vetor a partir do termo i , isto em ambas as formas.

e) $a+i$ é o endereço do i -ésimo elemento após a ;

Verdade.

f) $pa[i]$ é idêntico a $*(pa + i)$;

Verdade, se quisermos pegar um valor é indiferente uma das duas formas.

g) $pa = a$ é uma operação válida;

Verdade, pois como dito anteriormente estamos pegando o endereço do vetor a partir do primeiro termo.

h) $pa++$ é uma operação válida;

Verdade, pois estamos indo para o próximo elemento do endereço do nosso ponteiro.

i) $a = pa$ é uma operação válida;

Falso, pois um vetor não pode ter seu endereço modificado.

j) $a++$ é uma operação válida;

Falso, pois vetores não são uma variável.

24.

a) void main() /* esse programa está errado */

```
{  
int x, *p;  
x = 10;  
*p = x;  
}
```

O que está errado no programa é o fato de estarmos igualando o valor do ponteiro a x ($*p=x$), o que faz o ponteiro apontar para algum lugar aleatório, visto que não pegamos nenhum endereço para o ponteiro. Vale ressaltar que o valor de x será armazenado em um endereço qualquer do ponteiro.

O que deveria ser feito era " $p=&x$ " porque aí estaríamos pegando o endereço de x e consequentemente apontando para este endereço.

b)

void main() /* esse programa está errado */

```
{  
int x, *p;  
x = 10;  
p = x;
```

```
printf("%d", *p);  
}
```

O que está errado é a atribuição “p=x”, onde nela não pegamos o endereço de x para o ponteiro, ela só seria valida se x fosse um vetor, o que não é. Deveria ser feito “p=&x”.

27.

```
#include <stdio.h>  
  
int num;  
  
int func(int a, int b)  
{  
    a = (a+b)/2; /* Qual e o valor de a após a atribuição?      a = 25 */  
    num -= a;  
    return a;  
}  
  
main()  
{  
    int first = 0, sec = 50;  
    num = 10;  
    num += func(first, sec);  
    /* Qual e o valor de num, first e sec */  
    /* antes e depois da atribuicao? */  
    Antes: Num = 10, First = 0 e sec = 50  
    Depois: Num = 10, First = 0 e sec = 50  
  
    printf("\n\nConfira! num = %d\tfirst = %d\tsec = %d",num, first, sec);  
}
```

35.

```
#define max(A,B) ((A>B) ? (A):(B))  
#define min(A,B) ((A<B) ? (A):(B))
```

```
#define sqr(x) x*x          /* Ao se realizar esta operação dentro do macro o  
programa virá a apresentar problemas dependendo das variáveis inseridas.  
O que deveria ser feito para evitar isto seria inserir parênteses dentro das  
operações a serem realizadas (sqr(X) (X)*(X)). */
```

```
int main() {  
    int a = 10, b = 50, minimo, maximo, quad;  
    int z = 5;  
    minimo = min(a,b);  
    maximo = max(a++,b++);    /* Ao se realizar este tipo de atribuição a  
macro teremos problemas ao final, onde a variável maior terá um  
incremento de +2 em seu valor final, o que condiz com um erro  
(((a++)>(b++)) ? (a++):(b++)) */  
    quad = sqr(z+1);  
    return 0;  
}
```

Será indiferente para macro, visto que a mesma não tem especificação de tipo e trabalha de forma correta para todos os tipos de dados.